

Rust-Based Static Site Generator

Project Roadmap and Feature Plan

Project Overview

This document outlines the development roadmap and detailed feature plan for a high-performance, SEO-focused Static Site Generator written in Rust.

Roadmap

Phase 1 – Core Static Site Generation (Week 1–2)

Goal: Build the base system for markdown parsing, templating, and static HTML output.

- Set up Rust project using `cargo new`
- Read Markdown files from `/content`
- Convert Markdown to HTML using `pulldown-cmark`
- Apply templates using Tera
- Output HTML files to `/public`
- Copy static assets from `/static`

Phase 2 – SEO Essentials (Week 2–3)

Goal: Boost search engine visibility.

- Generate `sitemap.xml`
- Generate `robots.txt`
- Add `<title>` and `<meta>` tags
- Implement canonical URLs
- Add Open Graph / Twitter Card metadata
- Validate image `alt` attributes
- Generate structured data (JSON-LD)

Phase 3 – Developer Experience & Hot Reload (Week 3–4)

Goal: Enhance ease of development and real-time feedback.

- File watcher using `notify` crate
- CLI options: `build`, `serve`, `watch`, `deploy`
- Local development server
- Logging with `env_logger`

Phase 4 – Deployment Integrations (Week 5)

Goal: Easy deployment to static hosts.

- GitHub Pages support
- Netlify `deploy/export`
- Cloudflare Pages compatibility
- CLI `deploy` command

Phase 5 – Premium Features (Week 6–8)

Goal: Add advanced features and monetization.

- GUI using Tauri
- SEO audit CLI tool
- Blog support: tags, pagination
- RSS feed generator
- Theme and template system
- Multi-language (i18n) support

Detailed Feature Breakdown

1. Markdown Parsing

What: Convert Markdown to HTML.

```
use pulldown_cmark::{Parser, html::push_html};
let parser = Parser::new(&markdown_string);
let mut html_output = String::new();
push_html(&mut html_output, parser);
```

2. Template Rendering

What: Render content into layout templates using Tera.

```
let tera = Tera::new("templates/**/*.html")?;  
let rendered = tera.render("base.html", &context)?;
```

3. Sitemap Generator

What: Generates sitemap.xml.

```
use sitemap::writer::SiteMapWriter;  
let mut sitemap = SiteMapWriter::new(File::create("public/sitemap.xml")?);
```

4. robots.txt Generator

What: Basic crawler instructions.

```
fs::write("public/robots.txt", "User-agent: *\nAllow: /\nSitemap: ...")?;
```

5. Canonical URL Injection

What: Prevent duplicate content.

```
<link rel="canonical" href="https://yoursite.com/page.html">
```

6. SEO Meta Tags

What: Adds metadata for search engines.

```
<title>{{ page.title }}</title>  
<meta name="description" content="{{ page.description }}">
```

7. Open Graph & Twitter Card

What: Link previews on social platforms.

```
<meta property="og:title" content="{{ page.title }}">  
<meta name="twitter:card" content="summary">
```

8. Image Alt Tag Checker

What: Warns for missing alt tags.

How: Scan HTML output for tags without alt.

9. Structured Data (JSON-LD)

What: Helps Google understand page content.

```
<script type="application/ld+json">
{ "context": "...", "type": "Article", "headline": "{{ page.title
  }}" }
</script>
```

10. Live Reload / File Watcher

What: Auto-rebuild on file changes.

How: Use `notify crate` to monitor file system.

11. Deployment Support

What: Push output to GitHub Pages, Netlify, etc.

How: Use CLI + API keys or shell scripting.

12. SEO CLI Audit Tool (Pro)

What: Checks:

- Keyword density
- Readability (Flesch)
- Missing meta tags

13. RSS Feed Generator

What: Supports blog content syndication.

How: Generate RSS XML using post metadata.

14. Themes & Templates

What: Easy swapping of layouts/styles.

How: Load from a `/themes` directory with config.

15. Internationalization (i18n)

What: Multi-language site generation.

How: Organize content by language folders + use config.